Todo Danza: checklist de calidad (Tipos de datos básicos, clases, métodos, atributos y variables)

A. Rutinas o métodos

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
A-01	**Razón de existencia**	Cada método tiene un porqué es implementado.	Sí	Sí, cada método cumple una función importante en la estructura de nuestro programa.
A-02	**Propósito bien definido**	Cada método tiene un propósito bien definido.	Sí	Los métodos tienen un propósito bien definido, pertinente a la clase.
A-03	**Nombre bien definido**	El nombre del método dice exactamente lo que hace.	Sí	Los métodos hacen únicamente lo que describe en su nombre; ni más ni menos.
A-04	**Nombres con verbos**	Los nombres de los métodos usan verbos.	Sí	Los métodos están nombrados a partir de verbos (acciones) que semánticamente son correctos dada la acción que realizan y el objeto que afectan.
A-05	**Poca dependencia**	El método depende poco del resto del programa.	Sí	El método al tener una sola responsabilidad bien definida necesita la cantidad mínima del resto de elementos del programa para funcionar correctamente.
A-06	**Parámetros con sentido**	Los parámetros dentro del método tienen sentido en conjunto.	Sí	Los parámetros cumplen la función de describir la abstracción del objeto de la vida real.
A-07	**Orden lógico y constante**	Los parámetros siguen un orden lógico y siempre el mismo.	Sí	Sí, los parámetros siguen un mismo formato a lo largo de todas las clases.
A-08	**Pocos parámetros**	El método tiene pocos parámetros, de preferencia no más de 7.	Sí, en su mayoría	Hay algunas clases que tienen 8 atributos, pero la mayoría no pasan de 5.
A-09	**Partes separadas**	Las partes que se pueden separar en su propio método están separadas.	Sí	Sí, no hay ningún método que esté ejecutando dos funciones al mismo tiempo. Cada método hace solo una cosa con el fin de hacer que el sistema sea lo menos complejo posible.

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
A-10	**Longitud según lo que hace**	La longitud del método depende de lo que hace.	Sí	La longitud de los métodos en general no es muy larga, ya que hemos delimitado de manera apropiada sus responsabilidades y capacidades a lo largo del programa.
A-11	**Parámetros usados**	Se usan todos los parámetros del método.	Sí.	Todos los parámetros que se le pasan a cada método sirven para que pueda ejecutar su función de manera apropiada.
A-12	**Valor devuelto válido**	Si es una función, devuelve un valor válido en todos los casos.	Sí	Las funciones tienen validaciones considerando un amplio espectro de casos que sirven para evitar errores en tiempo de compilación.
A-13	**Convención para operaciones parecidas**	Las operaciones parecidas se nombran con la misma convención.	Sí	El nombramiento de operaciones parecidas (como las operaciones CRUD de la capa DAO) siguen convenciones que las hacen consistentes y fáciles de usar.

B. Variables

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
B-01	**Parámetros validados**	Los parámetros de entrada de los métodos se validan cuando es necesario.	Si	Los parámetros que se le pasan a una rutina se validan conforme a las reglas establecidas en el negocio (que el stock no pueda adoptar valores negativos por ejemplo).
B-02	**Declaración cerca del uso**	Las variables se declaran cerca de donde se usan por primera vez.	Si	Las variables se declaran cerca del lugar en el que se usan dentro de la implementación del programa.
B-03	**Inicialización antes de usarse**	Las variables se inicializan antes de usarse.	Sí	Las variables siempre se inicializan antes de usarse; nunca cuando se van a usar.

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
B-04	**Contadores y acumuladores**	Los contadores y acumuladores se inicializan y reinician bien.	Sí	Los acumuladores dentro del código se declaran cerca del código donde se usarán, se inicializan en el ciclo y se reinician una vez que ha terminado de ejecutarse el mismo.
B-05	**Alcance mínimo**	Cada variable tiene el menor alcance posible.	Sí	Las variables tienen el alcance justo que necesitan para los lugares en los que se van a usar (por ejemplo, los contadores no están declarados a nivel global).
B-06	**Usos cercanos**	Los usos de una variable están cerca unos de otros.	Sí	Los usos de la variable están cerca unos de otros porque los lugares en los que se utiliza están bien definidos y no hay necesidad de usarla desperdigada a lo largo de todo el código.
B-07	**Vida corta**	Las variables no viven más tiempo del necesario.	Sí	Las variables se declaran y se descartan únicamente en el tiempo en el que es necesario utilizarlas.
B-08	**Sin variables globales**	Se evitan las variables globales innecesarias.	Sí	No usamos variables globales en ninguna parte del código ya que cada módulo se comunica con el inmediatamente con el anterior y nuestras rutinas y variables tienen responsabilidades bien definidas
B-09	**Sin variables sin usar**	No hay variables sin usar.	Sí	Todas las variables que se crean son usadas en algún punto del programa.
B-10	**Propósito claro**	Cada variable tiene un propósito claro.	Sí	Todas las variables cumplen representar un atributo de una abstracción de un objeto de la vida real.
B-11	**Sin reutilización**	Una variable no se reutiliza para cosas distintas.	Sí	Las variables cumplen una sola función bien definida dentro del código y por ende no hay necesidad de que sean reusadas.
B-12	**Valores que cambian en configuración**	Los valores que pueden cambiar se leen de una configuración y no se escriben en el código.	Sí	Los valores cambian según la configuración que se tenga.

C. Nombres de variables

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
C-01	**Nombres adecuados**	Los nombres dicen bien lo que representan.	Sí	Los nombres que utilizamos definen de manera precisa lo que hacen las variables.
C-02	**Conceptos del problema**	Los nombres usan conceptos del problema, no detalles técnicos.	Sí	Los nombres describen lo que una variable como tal realiza, los detalles de la implementación quedan en un segundo plano al usarla.
C-03	**Longitud adecuada**	La longitud de los nombres es adecuada.	Sí	La longitud que le asignamos a los nombres es acertada porque es dependiendo de su scope, relevancia y complejidad.
C-04	**Sin nombres vagos**	Se evitan los nombres vagos o poco entendibles.	Sí	Todos los nombres son específicos.
C-05	**Índices descriptivos**	Los índices de ciclos anidados o complejos tienen nombres descriptivos.	N/A	No aplica
C-06	**Variables temporales con sentido**	Las variables temporales tienen nombres con sentido cuando su propósito no es obvio.	N/A	No aplica
C-07	**Booleanas legibles**	Las variables booleanas se leen bien cuando valen verdadero.	Sí	Aplicamos buenas prácticas a la hora de declarar variables booleanas.
C-08	**Enumeraciones claras**	Los nombres de las enumeraciones dejan claro cada estado.	Sí	Los valores que albergan los enums son coherentes con sus nombres y pertinentes dentro del contexto en el que se usan.
C-09	**Constantes por significado**	Las constantes se nombran por lo que significan, no por su valor.	N/A	No hay constantes definidas.

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
C-10	**Elementos parecidos**	Los elementos parecidos se nombran de forma parecida.	Sí	Los elementos que se parecen tienen similitudes a la hora del nombramiento como una convención informal que apunta hacia la legibilidad del código.
C-11	**Misma convención**	Todo el proyecto usa la misma convención de nombres.	Sí	
C-12	**Sin abreviaturas innecesarias**	Se evitan las abreviaturas innecesarias y los nombres que se confunden fácil.	Sí	Nunca usamos abreviaturas con el fin de “escribir más rápido”, siempre los nombres de las clases, variables y rutinas de modo que sea legible por sobre que sea práctico.
C-13	**Abreviaturas documentadas**	Las abreviaturas que se usan están documentadas.	N/A	No usamos ninguna abreviatura que tenga necesidad de ser documentada.
C-14	**Calificadores al final**	En los nombres de valores calculados, el calificador va al final.	Sí	Todos los calificadores los pusimos al final.

D. Tipos de datos básicos

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
D-01	**Sin números mágicos**	No hay números mágicos en el código.	Sí	No tenemos números mágicos en el código. Cualquier cantidad está especificada mediante una constante.
D-02	**Constantes con nombre**	Los valores constantes importantes tienen nombre.	Sí	Las constantes tienen nombre para evitar el uso de “números mágicos” y promover la legibilidad y posterior mantenibilidad del código.
D-03	**Compilación sin advertencias**	El código compila sin advertencias.	Sí	El código compila de manera correcta sin saltar ninguna advertencia.
D-04	**Desbordamiento de enteros**	Se considera el desbordamiento de enteros.	Sí	No se desbordan los enteros por la escala en la que estamos ejecutando el programa.

ID	Aspecto por evaluar	Descripción	¿Cumple?	Justificación
D-05	**Precisión de decimales**	Se tienen en cuenta la precisión y el redondeo de los números decimales (float y double).	Sí	Tomamos en cuenta que los tipos de datos que usamos no tengan errores, principalmente a la hora de hacer operaciones con dinero.
D-06	**Booleanos para sí o no**	Se usan booleanos para condiciones de sí o no.	Sí	Usamos el tipo de datos booleano para las operaciones disyuntivas (de sí y no estricto) con el fin de mejorar la legibilidad y aprovechar las facilidades que ofrecen los lenguajes con este tipo de datos.
D-07	**Enumeraciones para valores fijos**	Se usan enumeraciones cuando hay un conjunto fijo y conocido de valores.	Sí	Los enums son usados en valores como el tipo de precio (mayoreo o público) o el estado de una venta (completada, pendiente o cancelada).
D-08	**Dinero sin errores de precisión**	Los cálculos con dinero evitan los problemas de precisión de los decimales.	Sí	Los cálculos de dinero se hacen con las operaciones apropiadas para evitar errores de precisión debido al modo en el que se almacenan en las computadoras como valores binarios.
D-09	**Valores inválidos en enumeraciones**	Las pruebas con enumeraciones verifican valores inválidos.	No	Todos los valores de los enums son válidos en las pruebas.
D-10	**Un tipo por cada dato que cambia**	Se usa un tipo distinto por cada clase de dato que puede cambiar.	Sí	Sí se usa un tipo distinto por cada dato que pueda cambiar.
D-11	**Tipos del mundo real**	Los nombres de los tipos se orientan a las entidades reales que representan.	Sí	Los tipos describen objetos del mundo real, abstraídos en código.